

115 學年度資訊學科能力競賽臺南一中校內複選

題解

Otc 87. 2026

D. Merge, Rebase, Squash!

D. Merge, Rebase, Squash! – Fun Fact



Force push 1



Force push 2

- 不要學 DB force push，真的會出事
- 這題其實是我在思考甚麼情況下 Merge 與 Rebase 都會 Conflict，甚麼情況下只有 Rebase 會 Conflict 想出來的

D. Merge, Rebase, Squash! – 題目敘述

- v2.0 上有 H 個 hunk；第 h 個內容是 U_h ，解 conflict 的成本是 w_h 。
- feat/fancy 有 N 個 commits；第 i 個把 hunk x_i 改成 v_i 。
- 在相鄰 commits 間放 $K - 1$ 個切點，得到恰好 K 個 squash commits，再依序 rebase。

D. Merge, Rebase, Squash! – Conflict

一個 patch $\text{old} \rightarrow \text{new}$ 遇到目前內容 current 時：

$$\text{conflict} \iff \text{current} \notin \{\text{old}, \text{new}\}.$$

目標：選擇切點，使 conflict 的總成本最小。

D. Merge, Rebase, Squash! – 記號

- $S_i[h]$: 執行完前 i 個 commits 後, feat/fancy 上 hunk h 的內容。
- f_h : hunk h 第一次被修改的 commit ; 若從未修改則不存在。

D. Merge, Rebase, Squash! – Squash 後的 patch

區間 $(j, i]$ squash 後，hunk h 的 patch 是

$$S_j[h] \longrightarrow S_i[h].$$

題目保證 hunk 不會被改回 0，所以包含 f_h 的 squash commit 一定真的會修改 h 。

D. Merge, Rebase, Squash! – 第一次修改

考慮包含 f_h 的第一個 squash commit，假設其右端點為 i ：

$$0 \longrightarrow S_i[h].$$

此時 v2.0 的內容仍是 U_h 。又因 $U_h \neq 0$ ，所以

$$h \text{ 發生 conflict} \iff S_i[h] \neq U_h.$$

D. Merge, Rebase, Squash! – 不會再次 conflict

解完後目前內容變成 $S_i[h]$ 。之後每個 squash commit 的 old 都等於目前內容，因此不可能再次 conflict。

上面很抽象，可以感性理解一下，顯然你 Merge 時只要在某個 commit 解決衝突後，後面的 commit 就不可能衝突了。

因為後面都是你自己的 commit，你不會 commit 會衝突的東西上去，這樣要怎麼 commit 成功。

結論

hunk h 的成本，只由「包含第一次修改 f_h 的那一段」決定。

D. Merge, Rebase, Squash! – 一段 squash commit 的成本

定義 $C(j+1, i)$ 為把 commits $j+1, \dots, i$ squash 成一段時，這一段新產生的 conflict 成本。

它只需要計算第一次修改落在這段內的 hunks：

$$C(j+1, i) = \sum_{\substack{h: j < f_h \leq i \\ S_i[h] \neq U_h}} w_h$$

因為每個 hunk 只會在包含 f_h 的那段被計算一次，所以所有段的 C 相加，恰好就是整次 rebase 的總成本。

D. Merge, Rebase, Squash! – 子任務

子任務	限制	分數
1	$N, H \leq 15$	8
2	$K = 1$	8
3	$K = N$	8
4	每個 hunk 最多修改一次	10
5	$N \leq 1000$	26
6	無額外限制	40

完整限制： $N, H \leq 2 \times 10^5$ ， $K \leq 30$ 。

D. Merge, Rebase, Squash! – 子任務 1：枚舉切點

從 $N - 1$ 個空隙選出 $K - 1$ 個切點：

$$\binom{N - 1}{K - 1} \text{ 種分法.}$$

對每種分法直接模擬：

- 1 預先算出每個 commit 後的狀態 S_i 。
- 2 對每段 squash commit 產生 patch。
- 3 按題目的規則 rebase，累加 conflict 成本。

時間複雜度

$$O\left(\binom{N - 1}{K - 1}(N + KH)\right),$$

在 $N, H \leq 15$ 時足夠。

D. Merge, Rebase, Squash! – 子任務 2 : $K = 1$

全部 squash 成 $(0, N]$ 。答案為

$$\sum_{h: f_h \text{ 存在, } S_N[h] \neq U_h} w_h.$$

D. Merge, Rebase, Squash! – 子任務 3–4

- $K = N$ ：每個 commit 單獨套用，只有第一次修改可能 conflict。答案為

$$\sum_{h: f_h \text{ 存在}, v_{f_h} \neq U_h} w_h.$$

- **每個 hunk 最多修改一次**：不論怎麼切，包含那次修改的區間終點值都相同；答案同樣是 $\sum_{h: f_h \text{ 存在}, v_{f_h} \neq U_h} w_h$ 。

三者皆可在 $O(N + H)$ 完成。

D. Merge, Rebase, Squash! – 子任務 5：DP 定義

定義

$dp[k][i]$ = 前 i 個 commits 分成恰好 k 個非空段的最小成本.

初始條件：

$$dp[0][0] = 0, \quad dp[0][i] = +\infty \quad (i > 0).$$

D. Merge, Rebase, Squash! – 子任務 5：DP 轉移

枚舉最後一段前的切點 j ：

$$dp[k][i] = \min_{k-1 \leq j < i} (dp[k-1][j] + C(j+1, i)).$$

最終答案是 $dp[K][N]$ 。

D. Merge, Rebase, Squash! – 預處理：依 f_h 分桶

固定右端點 i ，把目前滿足 $S_i[h] \neq U_h$ 的 hunk，依照第一次修改位置 f_h 分桶：

$$B_i[t] = \sum_{h: f_h=t, S_i[h] \neq U_h} w_h.$$

每前進一個 commit，只會改變一個 hunk 所在桶子的值。

D. Merge, Rebase, Squash! – 預處理：計算 C

由成本公式可得

$$C(l, i) = \sum_{t=l}^i B_i[t].$$

再對 B_i 做一次後綴和，就能在 $O(N)$ 算出所有 $C(l, i)$ 。

- 預處理： $O(N^2 + H)$
- DP： $O(KN^2)$

D. Merge, Rebase, Squash! – 完整解：重新看一次轉移

固定 DP 層 k ，令

$$prev[j] = dp[k-1][j], \quad A_i[j] = prev[j] + C(j+1, i).$$

則

$$dp[k][i] = \min_{k-1 \leq j < i} A_i[j].$$

D. Merge, Rebase, Squash! – 需要的資料結構

從左到右掃描右端點 i 。我們需要維護所有切點 j 的 $A_i[j]$ ，並支援：

- 一段切點同時加上相同成本；
- 查詢合法切點 $[k - 1, i - 1]$ 的最小值。

這正是「區間加值、區間最小值」線段樹。

D. Merge, Rebase, Squash! – 一次 commit 影響哪些切點？

commit i 修改 hunk h ，令 $f = f_h$ 。

hunk h 會被 $C(j + 1, i)$ 計入的必要條件是

$$j < f.$$

因此當 $S_i[h] \neq U_h$ 的真假改變時，它只會同時影響

$$j = 0, 1, \dots, f - 1,$$

恰好是一段前綴。

預先替每個 commit 算出

$$right[i] = f_h - 1, \quad \text{delta}[i] = \text{成本變化},$$

掃到 i 時對 $[0, right[i]]$ 加上 $delta[i]$ 。

D. Merge, Rebase, Squash! – *delta* : 第一次修改

第一次修改 hunk h ($i = f_h$) :

$$right[i] = i - 1,$$

$$delta[i] = \begin{cases} +w_h, & v_i \neq U_h, \\ 0, & v_i = U_h. \end{cases}$$

D. Merge, Rebase, Squash! – *delta*：後續修改

不是第一次修改，令 $old = S_{i-1}[h]$ 、 $new = S_i[h]$ ：

$$right[i] = f_h - 1,$$

$$delta[i] = \begin{cases} +w_h, & old = U_h, new \neq U_h, \\ -w_h, & old \neq U_h, new = U_h, \\ 0, & \text{其餘情況.} \end{cases}$$

也就是只追蹤「目前內容是否不等於 U_h 」的狀態變化。

D. Merge, Rebase, Squash! – 完整演算法 (1/2)

對 $k = 1, 2, \dots, K$:

- 1 用 $prev[0], \dots, prev[N - 1]$ 建立線段樹。
- 2 依序掃描 $i = 1, 2, \dots, N$ 。
- 3 對 $[0, right[i]]$ 加上 $delta[i]$ 。

D. Merge, Rebase, Squash! – 完整演算法 (2/2)

4 若 $i \geq k$ ，查詢

$$cur[i] = \min_{k-1 \leq j < i} A_i[j] = \text{rangeMin}(k-1, i-1).$$

5 掃完後交換 $prev$ 與 cur 。

初始只有 $prev[0] = 0$ ，其餘皆為 $+\infty$ ；答案是 $prev[N]$ 。

D. Merge, Rebase, Squash! – 正確性

線段樹不變量

在某一層掃描完 commit i 後，葉節點 j 儲存的值恰為

$$prev[j] + C(j + 1, i).$$

每個 hunk 對 $C(j + 1, i)$ 的貢獻，只在它的狀態跨過 U_h 時改變，且只影響 $j < f_h$ ；因此 prefix range add 完整維護了不變量。

查詢 $[k - 1, i - 1]$ 就恰好枚舉所有合法的最後切點。

D. Merge, Rebase, Squash! – 複雜度

時間 $O(H + KN \log N)$, 空間 $O(H + N)$.

成本總和最大可達 2×10^{14} ，必須使用 64-bit integer。

G. Performance Analysis

G. Performance Analysis – Fun Fact

- 原本只是隨便想想拿去初賽，結果出完發現太難了倒了
- blame 用線代解，直接倒了

G. Performance Analysis – 題目敘述

給一張有 N 個 basic blocks、 M 條有向邊的 CFG。程式從入口 1 執行到出口 N ，總共執行 R 次。

- 在邊 e 安裝計數器需花費 c_e ，之後可直接知道流量 f_e 。
- 沒裝計數器的邊，只能利用每個點的 flow conservation 推導。
- 要讓所有邊的流量都能被**唯一確定**，並最小化安裝成本。

G. Performance Analysis – Flow conservation

普通點滿足

$$\sum_{e \in \text{in}(v)} f_e = \sum_{e \in \text{out}(v)} f_e,$$

入口與出口則分別多出、少掉 R 的流量。

G. Performance Analysis – 改成最大化省下的成本

令 F 為**沒有安裝計數器**的邊集合。

如果所有邊都裝計數器，成本是

$$C_{all} = \sum_{e \in E} c_e.$$

選擇不測量 F 可以省下

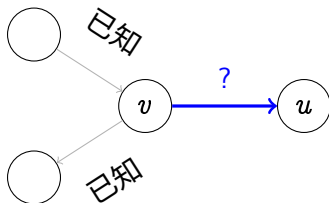
$$\sum_{e \in F} c_e.$$

所以問題變成：

在仍能唯一推導所有未知流量的前提下，
找出總權重最大的邊集合 F 。

G. Performance Analysis – F 是森林

先忽略邊的方向。考慮 F 中任意一棵樹，取一個葉節點 v 。



G. Performance Analysis – 拔掉葉節點

v 只剩一條未知邊，因此用 v 的流量平衡式即可算出它。

算完後刪掉這條邊，剩下的仍是森林；反覆拔葉子即可算完所有未知邊。

入口或出口也一樣，只要在平衡式中加入已知的 R 。

G. Performance Analysis – F 有環

忽略方向後，假設未知邊形成一個環。

沿著環選擇適當正負號，將各邊流量同時調整 ε ：

$$f_e \leftarrow f_e \pm \varepsilon.$$

每個環上節點的流入與流出變化會互相抵消，所有 flow conservation 方程仍成立，計數器讀值也完全不變。

G. Performance Analysis – 環使答案不唯一

題目的 $R \geq 2M$ 與可達性保證能讓每條環邊都有正整數流量，因此可取 $\varepsilon = 1$ ，得到另一組合法流量，無法唯一決定。

環也包含特殊情況

self-loop 自己就是一個環；兩條連接相同端點的重邊也能形成環。

G. Performance Analysis – 圖論轉換

綜合前兩頁：

所有未知流量可唯一確定 $\iff F$ 忽略方向後是森林

因此要找最大權重生成森林，最大化省下的成本。

G. Performance Analysis – 從森林到生成樹

題目保證每個 basic block 都在某條入口到出口的路徑上，所以忽略方向後整張圖連通；又因 $c_e > 0$ ，最佳森林一定能擴充成生成樹。

等價問題

找忽略方向後的**最大權重生成樹**。

G. Performance Analysis – 子任務

子任務	限制	分數
1	$M = N - 1$ ，第 i 條邊為 $i \rightarrow i + 1$	5
2	所有 $c_e = 1$	10
3	$M \leq 20$	15
4	$M = N$ ，且沒有 self-loop 或重複連接	15
5	每個點雙連通分量都是單邊或簡單環	20
6	無額外限制	35

完整限制： $N \leq 10^5$ ， $M \leq 2 \times 10^5$ 。

G. Performance Analysis – 鏈與 $c_e = 1$

- 鏈：所有邊本來就是一棵樹，全部都可以不裝；答案是 0。
- $c_e = 1$ ：生成樹能省下 $N - 1$ ，所以必須安裝

$$M - (N - 1) = M - N + 1$$

條邊，答案也是 $M - N + 1$ 。

G. Performance Analysis – $M \leq 20$

枚舉哪些邊不裝計數器，再用 DSU 檢查是否形成環；取合法集合中省下成本的最大值。

複雜度 $O(2^M M \alpha(N))$ 。

G. Performance Analysis – 單環圖

$$M = N :$$

連通無向圖恰有一個環。環上至少一條邊必須裝計數器，裝成本最小的那條即可；答案是環上最小 c_e 。

G. Performance Analysis – 仙人掌

仙人掌：

每個點雙連通分量不是單邊就是簡單環。每個環至少要裝一條邊，而且不同環的選擇互不影響；對每個環加上其中最小的 c_e 。

這兩個做法都在做同一件事：

每次破壞環時，盡量保留成本大的邊。

G. Performance Analysis – Maximum Kruskal (1/2)

- 1 計算 $C_{all} = \sum_e c_e$ 。
- 2 將所有邊依 c_e 由大到小排序。
- 3 用 DSU 維護忽略方向後的連通塊。

G. Performance Analysis – Maximum Kruskal (2/2)

- 4 若邊 (u, v) 的兩端尚未連通，就把它加入最大生成樹，合併兩端並累加 $saved += c_e$ 。

最後

$$answer = C_{all} - saved.$$

self-loop 的兩端本來就在同一個集合，因此一定會安裝計數器；重邊則最多選一條進入生成樹。

G. Performance Analysis – Kruskal 的正確性

Kruskal 由大到小考慮邊：只要加入後仍是森林，就保留這條高成本邊。

依生成樹的交換論證，最後得到的 *saved* 是所有生成樹中的最大權重。

G. Performance Analysis – 整體正確性

又因：

- 未測量邊是森林 $\Rightarrow$ 所有流量可由拔葉子唯一算出；
- 未測量邊有環 $\Rightarrow$ 可以沿環調整，流量不唯一；

所以 $C_{all - saved}$ 正是最小安裝成本。

G. Performance Analysis – 複雜度

時間 $O(M \log M)$, 空間 $O(N + M)$.

R 不影響要裝哪些邊，演算法中不會使用；成本總和要用 64-bit integer。